

Pricing Refactor Regression

This assignment consists of a single data file comparing pricing engine output before and after a refactor.

Files

`pricing_diff.csv` : one row per order, computed twice, once with the old pricing code (`v1_total`) and once with the new, refactored pricing code (`v2_total`) that shipped last week.
Columns:

column	meaning
<code>order_id</code>	unique identifier
<code>weight_kg</code> , <code>distance_km</code>	order inputs
<code>category</code>	one of <code>standard</code> , <code>electronics</code> , <code>fragile</code> , <code>books</code> , <code>clothing</code> , <code>food</code>
<code>express</code>	<code>True</code> / <code>False</code>
<code>coupon</code>	<code>"SAVE10"</code> or empty
<code>v1_total</code>	price computed by the old code
<code>v2_total</code>	price computed by the new code

The Problem

Last week's release refactored the pricing engine. Two things you already know going in:

- The refactor **intentionally** changed the per-kg rate for the `books` category (this was a deliberate, approved pricing change, not a bug).
- The refactor also changed the order in which some numbers are added up internally, so **tiny floating-point differences (a cent or two) between `v1_total` and `v2_total` are expected on essentially every order** and are not meaningful on their own.

Now customer support is getting complaints from customers who say they were charged noticeably more than expected, not by a cent or two. Something beyond the intentional `books` change and the harmless rounding noise is going wrong somewhere in the refactor. Find it.

Questions

1. If you just count rows where `v2_total != v1_total` , how many rows is that? Explain in 1-2 sentences why this number is **not** a useful answer to "how many orders are actually affected by a real bug."
2. What is the exact number of orders affected by a genuine pricing regression (i.e., NOT the documented `books` change, and NOT floating-point noise)? What input conditions (category, `express` , etc.) do all of the affected orders share?
3. What is the total dollar amount customers were overcharged (or undercharged) across those affected orders? Report the exact number.
4. As a sanity check: what is the average `v1_total` -vs- `v2_total` difference for orders that are NOT affected by the bug and NOT in the `books` category? Use this to justify that your answer to Q2 is a real, distinct pattern and not just "everything is a little different."
5. **Bonus:** What do you think the actual code-level bug is? You don't have the pricing code, just describe the bug in plain English based on the pattern you found.

Submit

- `ANSWERS.md` with your answers to all 5 questions.
- `answers.json` :

```
{
  "q1_naive_nonzero_count": 0,
  "q2_affected_count": 0,
  "q2_affected_category": "",
  "q3_total_overcharge": 0.0,
  "q4_baseline_mean_abs_diff": 0.0
}
```

- Any scripts or code you used.
- A short section (5–10 bullet points) describing your investigation process, including dead ends.

Ground rules

- You may use any language or tools, including AI assistants. You should be able to walk through your analysis and reasoning live.
- Do not submit `pricing_diff.csv`.
- Reading the CSV only requires a standard library or `pandas`; no other setup needed.